

Documentación técnica del proyecto

1. Arquitectura MVVM utilizada

El proyecto sigue el patrón MVVM (Model–View–ViewModel), separando la lógica de negocio, la lógica de presentación y la interfaz gráfica.

- Model: gestiona el acceso a datos y las operaciones CRUD.
- View: contiene las ventanas WPF y los elementos visuales.
- ViewModel: actuaría como intermediario entre View y Model, exponiendo propiedades y comandos. En este proyecto, parte de esa lógica se resuelve desde Controller, por lo que el rol de ViewModel está integrado en esa capa.

2. Diagrama de capas

Capas principales del proyecto:

- View: interfaz WPF.
- Controller: lógica de negocio y coordinación.
- Model: acceso a datos y operaciones con la base.
- Informes: generación de reportes.
- Test: pruebas unitarias e integración.

Flujo principal:

View → Controller → Model → Base de datos

3. Descripción de carpetas del proyecto

/View: ventanas WPF (VinilosView, ArtistasView, etc.).

/Controller: lógica de negocio para cada entidad (VinilosController, ArtistasController, etc.).

/Model: acceso a datos y consultas (VinilosModel, ArtistasModel, etc.).

/Informes: dataset y vistas de informes.

/Test: pruebas unitarias e integración.

4. Explicación de componentes

Models

Representan la capa de datos. Contienen métodos para listar, insertar, actualizar y eliminar información.

Ejemplo: VinilosModel gestiona operaciones sobre la tabla de vinilos.

ViewModels

En una arquitectura MVVM completa, esta capa gestionaría el binding de datos y los comandos. En este proyecto esa responsabilidad se resuelve directamente desde los controladores, por lo que la capa ViewModel no está separada físicamente.

Commands

En WPF, los comandos deberían encapsular acciones del usuario. En este proyecto las acciones se gestionan directamente desde los eventos de botones en las vistas.

Acceso a datos

La capa Model se comunica directamente con la base de datos y devuelve resultados en DataTable, que son utilizados por Controller y mostrados en View.